哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

高级语言程序设计
High-level Language Programming

Lecture 11 Polymorphism

Weizheng Zhang (zhangweizheng@hit.edu.cn)
School of Information Science and Technology

Polymorphism

Course Overview

- Polymorphism: basic idea
- Templates
- Virtual function
- Abstract base classes

Try it yourself on https://www.onlinegdb.com

11.1 What is polymorphism

The word ***polymorphism*** is derived from a Greek word meaning "many forms".

Polymorphism is one of the most important features in object-oriented programming and refers to the **ability of different objects to respond differently to the same command** (or 'message' in object-oriented programming terminology).

11.1 What is polymorphism

- For example, the command '**open**' means different things when applied to different objects.
  - Opening a bank account is very different from opening a window, which is different from opening a window on a computer screen.

  - The command ('**open**') is the **same**, but depending on what it is applied to (the object) the **resultant actions are different**.

11.1 What is polymorphism

```
6  class advanced_computer
7  {
8  public:
9  void hello()
10 {
11   cout << "Hello from the Advanced Computer" << endl ;
12 }
13 } ;
14
15 class simple computer
16 {
17 public:
18 void hello()
19 {
20   cout << "Hello from the Simple Computer" << endl ;
21 }
22 } ;
23
```

```
24 main()
25 {
26   advanced_computer HAL ;
27   simple_computer PC ;
28   HAL.hello() ;
29   PC.hello() ;
30 }
```

Polymorphism:
The same command to different objects invokes different actions.

11.1 What is polymorphism

- Polymorphism can be divided into two broad categories: ***static*** (or compile-time) and ***dynamic*** (or run-time) *polymorphism*.
- Static polymorphism occurs when the program is being compiled, dynamic polymorphism when the program is actually running.
  - *static* or *early binding:* normally, which function a call refers to is made at compile time. The compiler knows which function to call based on the object that calls it.
  - *dynamic* or *late binding*: it is left until run-time to determine which function should be called. This decision is based on the object making the call.

11.2 Static and dynamic polymorphism

- C++ has three **static polymorphism** mechanisms:
  - **function overloading （Lecture 8 Functions）**
  - operator overloading
  - Templates

- **Dynamic polymorphism** of C++: **virtual function**

function overloading

```
int sum_array ( const: int array [] , int no_of_elements )
{
    int total = 0;

    for(int index = 0 ; index < no_of_elements ; index ++ )
        total += array[index] ;
    return total;
}

int sum_array ( const int array[][2] int no_of_rows )
{
    int total = 0;

    for(int row = 0 ; index < no_of_rows ; row ++ )
    {
        for(int col = 0 ; col < 2 ; col++ )
            total += array[row][col] ;
    }
    return total;
}
```

Functions with the same names and different parameter lists mean function overloading.

11.2 Static and dynamic polymorphism

- Programmer can use some operator symbols to define special **member functions** of a class, providing convenient notations for object behaviors

- The + operator, for example, is appropriate for strings as well and is taken to mean concatenation. This means that **the operator + has a different meaning for numeric data types than for string data types**.

- When you create a new class you can redefine or overload existing operators such as +, - , *, /, %, etc. (Existing operators can be overloaded for specific use in a class by writing operator functions for the class)

11.2 Static and dynamic polymorphism

- The C++ built-in arithmetic (+,*,/etc.) and relational operators (>,<,==,!=)work with built-in data types (int, float, etc.).

- Not all the built-in operators can be used with every data type: for example, multiplication is not a valid operation for strings and % can only be used with integer data types.

11.2 Static and dynamic polymorphism

```
2  // Demonstration of an overloaded + operator.
3  #include <iostream>
4  using namespace std;
5
6  class time24 // A simple 24-hour time class.
7  {
8      public:
9          time24( int h = 0, int m = 0, int s = 0 ) ;
10         void set_time( int h, int m, int s ) ;
11         void get_time( int& h, int& m, int& s ) const ;
12         time24 operator+( int secs ) const ;
13     private:
14         int hours ;   // 0 to 23
15         int minutes ; // 0 to 59
16         int seconds ; // 0 to 59
17 } ;
```

```
18 // Constructor.
19 time24::time24( int h, int m, int s ) : hours( h ), minutes( m ), seconds( s )
20
21 // Mutator function.
22 void time24::set_time( int h, int m, int s )
23 {
24    hours = h ; minutes = m ; seconds = s ;
25 }
26
27 // Inspector function.
28 void time24::get_time( int& h, int& m, int& s ) const
29 {
30    h = hours ; m = minutes ; s = seconds ;
31 }
```

- The prototype for the overloaded + operator is on line 12.

11.2 Static and dynamic polymorphism

```
32 // Overloaded + operator.
33 time24 time24::operator+( int secs ) const
34 {
35    // Add secs to class member seconds and calculate the new time.
36    time24 temp ;
37    temp.seconds = seconds + secs ;
38    temp.minutes = minutes + temp.seconds / 60 ;
39    temp.seconds %= 60 ;
40    temp.hours = hours + temp.minutes / 60 ;
41    temp.minutes %= 60 ;
42    temp.hours %= 24 ;
43    return temp ; // Return the new time.
44 }
```

- define the overloaded + operator for adding an integer number of seconds to a time 24 object and returning the result.

11.2 Static and dynamic polymorphism

```
45 main()
46 {
47     int h, m, s ;
48     time24 t1( 23, 59, 57 ) ; // t1 represents 23:59:57
49     time24 t2 ;
50     t2 = t1 + 4 ; // t2 should now be 0:0:1
51     t2.get_time ( h, m, s ) ;
52     cout << "Time t2 is " << h << ":" << m << ":" << s << endl ;
53 }
```

- it can be called like any other function; line 50 can also be written as: t2 = t1.operator+(4) ;

11.2 Static and dynamic polymorphism

- The following rules apply to operator overloading:
  - New operators cannot be invented. For example, it is not possible to overload <> to mean ‘not equal to’.
  - The overloaded operator must have the same number of operands as the corresponding predefined operator. For example, an overloaded equivalent == must have two operands.
  - The priority of an overloaded operator remains the same as its corresponding predefined operator. For example, regardless of whether * is overloaded or not, it will always have a higher precedence than + and -.
  - Overloaded operators cannot have default arguments.
  - The operators for the built-in data types, e.g. int, float and so on, cannot be redefined.

11.3 Template

- A ***template*** is a **framework** for generating a **class** or a **function**.
  - *Instead* of explicitly specifying the **data types** used in a class or a function, **parameters are used**.
  - When actual data types are assigned to the parameters, the class or function is generated by the compiler.

11.3 Template

- A function to find the maximum of two **integer** values

```
int maximum( const int n1, const int n2 )
{
  if ( n1 > n2 )
    return n1 ;
  else
    return n2 ;
}
```

- A function to find the maximum of two **floating-point** values a nearly identical

```
float maximum( const float n1, const float n2 )
{
  if ( n1 > n2 )
    return n1 ;
  else
    return n2 ;
}
```

The reason two different functions are required is simply that the function header is different in the two functions.

11.3 Template

- **Function templates** allow both of these functions to be replaced by a single function in which the parameter data types are replaced by a parameter T (for type) giving a generic or type-less function that can be used for all data types.

```
3  #include <iostream>
4  using namespace std ;
5
6  template <typename T>
7  T maximum( const T n1, const T n2 )
8  {
9    if ( n1 > n2 )
10     return n1 ;
11   else
12     return n2 ;
13 }
14
```

T is called the **type parameters**. T is normally used, but any valid name can be used.

11.3 Template

- A **function template** declaration consists of the keyword template and a list of one or more data type parameters.

- The data **type parameter** is preceded by either the keyword class or the more meaningful keyword **typename**.

- **Type parameters** are enclosed within angle brackets < and >. When multiple data type parameters are used, they are separated by commas.

11.3 Template

```
6   template <typename T>
15  main()
16  {
17    char c1 = 'a', c2 = 'b' ;
18    int i1 = 1, i2 = 2 ;
19    float f1 = 2.5, f2 = 3.5 ;
20
21    cout << maximum( c1, c2 ) << endl ; //   a maximum() for chars
22    cout << maximum( i1, i2 ) << endl ; //   a maximum() for ints
23    cout << maximum( f1, f2 ) << endl ; //   a maximum() for floats
24  }
```

This process is called *instantiation of the template* and the result is *a* conventional function generated by the compiler.

```
template <typename T>
T maximum( const T n1, const T n2
{
  if ( n1 > n2 )
    return n1 ;
  else
    return n2 ;
}
```

Line 21 has the effect of replacing the type parameter *T* in line 6 with the data type of the arguments c1 and c2, i.e. char. The compiler generates a function with the prototype.

11.3 Template

- Instead of using three separate functions in the program, **a single function template is used.**
  - The program **source file is smaller**, but since the compiler generates three separate functions from the template, **the executable file size remains the same.**
- Template definitions can be placed in a header file

```
2  // Demonstration of including a function
3  #include<iostream>
4  #include "maximum.h"
5  using namespace std ;
6
7  main()
8  {
9     char c1 = 'a', c2 = 'b' ;
10    int i1 = 1, i2 = 2 ;
11    float f1 = 2.5, f2 = 3.5 ;
12
13    cout << maximum( c1, c2 ) << endl ;
14    cout << maximum( i1, i2 ) << endl ;
15    cout << maximum( f1, f2 ) << endl ;
16 }
```

11.3 Template

- Function templates to sort the elements of an array

```
4  #include <iostream>
5  #include "swap.h"
6  #include "bubble.h"
7  #include "show_array.h"
8  using namespace std ;
9
10 main()
11 {
12   int int_array[] = { 23, 4, 12, -9, 55 } ;
13   float float_array[] = { 4.2, 78.8, 77.3, 1.2 } ;
14
15   bubble_sort( int_array, 5 ) ;
16   show_array( int_array, 5 ) ;
17   bubble_sort( float_array, 4 ) ;
18   show_array( float_array, 4 ) ;
19 }
```

Header files containing functions template.

Use of function templates with different data types.

```
1  // Declaration of the bubble function template.
2  #ifndef BUBBLE_T_H
3  #define BUBBLE_T_H
4
5  template <typename T>
6  void bubble_sort( T array[], int n )
7  // Purpose : Sorts an array using the bubble sort algorithm.
8  // Parameters: The array to sort and
9  //             The number of elements in the array.
10 {
11   bool swapping = true ;
12   while( swapping )
13   {
14     swapping = false ;
15     for ( int i = 0 ; i < n-1 ; i++ )
16     {
17       if ( array[i] > array[i+1] )
18       {
19         swapping = true ;
20         swap_values( array[i], array[i+1] ) ;
21       }
22     }
23   }
24 }
25
26 #endif
```

```
1  // Declaration of the swap function template.
2  #ifndef SWAP_T_H
3  #define SWAP_T_H
4
5  template <typename T>
6  void swap_values( T &x, T &y )
7  // Purpose : Swap two values.
8  // Parameters: Two values of any data type.
9  {
10   T temp = x ;
11   x = y ;
12   y = temp ;
13  }
14
15 #endif
```

```
1  // Declaration of the show_array function template.
2  #ifndef SHOW_ARRAY_T_H
3  #define SHOW_ARRAY_T_H
4
5  using namespace std ;
6  template <typename T>
7  void show_array( T array[], int n )
8  // Purpose : Displays the elements of an array.
9  // Parameters: array[] - the array to display.
10 //             n - the number of elements in the array.
11 {
12   for ( int i = 0 ; i < n ; i++ )
13   {
14     cout << array[i] << ' ' ;
15   }
16   cout << endl ;
17 }
18 #endif
```

As with previous uses of header files, the preprocessor statements are used in each of these header files to prevent the header file from being inadvertently included more than once in a program.

11.3 Template

- Templates can be also used for generating entire C++ classes.
  - templates allow one class to be written for **all data types**

- Class templates are also known as *parameterized types*. A **parameterized type** is a data type defined in terms of other data types, some of which are unspecified.

- To create templates: 1) **write a non-template data type** specific version of the function or class. 2) when the data type specific version is working satisfactorily, **replace the specific data types with template parameters**.

11.3 Template

- A stack can be defined as a class template with a type parameter that specifies the data type of the items to be stored on the stack and an integer variable to indicate the current top of the stack.

- The definition of a stack class template is stored in a header file stack.h

```
1  // Declaration of stack class template.
2  #ifndef STACK_T_H
3  #define STACK_T_H
```

```
5  template <typename T>
6  class stack
7  {
8  public:
9    stack( int n = 10 ) ;
10   // Purpose   : Class constructor.
11   // Arguments: n - size of stack.
12   ~stack() ;
13   // Purpose   : Class destructor.
14   bool pop( T& data_item ) ;
15   // Purpose    : Remove an item from the stack.
16   // Parameters: data_item - data item on top of the stack is
17   //             copied to this reference.
18   // Returns    : true - operation successful.
19   //              false - operation failed.
20   bool push( const T& data_item ) ;
21   // Purpose    : Add an item to the stack.
22   // Parameters: data_item is placed on top of the stack.
23   // Returns    : true - operation successful.
24   //              false - operation failed.
25    inline int number_stacked() const ;
```

```
26    // Purpose   : Inspector function.
27    // Returns   : Number of items currently on the stack.
28     inline int stack_size() const ;
29    // Purpose   : Inspector function.
30    // Returns   : Size of the stack.
31 private:
32    int max_size ;
33    int top ;
34    T* data ;
35 } ;
36 // stack member functions
37 template <typename T>
38 stack <T>::stack( int n )
39 {
40    max_size = n ;
41    top = -1 ;
42    data = new T[ n ] ;
43 }
44
```

```
45 template <typename T>
46 stack <T>::~stack()
47 {
48   delete [] data ;
49 }
50
51 template <typename T>
52 bool stack <T>::push( const T& data_item )
53 {
54   if (  top  <  max_size -1 )
55   {
56     data[ ++top ] = data_item ;
57     return true ;
58   }
59   else
60     return false ;
61 }
62
63 template <typename T>
64 bool stack <T>::pop( T& data_item )
65 {
66   if ( top > -1 )
67   {
68     data_item = data [ top-- ] ;
69     return true ;
70   }
71   else
72     return false ;
73 }
74
75 template <typename T>
76 int stack <T>::number_stacked() const
77 {
78   return top + 1 ;
79 }
80
81 template <typename T>
82 int stack <T>::stack_size() const
83 {
84   return max_size ;
85 }
86
87 #endif
```

11.3 Template

- A reference to a class template must include its parameter list.
  - The definition of the member functions use stack<T> instead of just stack, which would be used if the class were not a template.

```
3  #include <iostream>
4  #include "stack.h"
5  using namespace std ;
6
7  main()
8  {
9     stack <int> i_stack ;
10    stack <char> c_stack( 5 ) ;
11    int i, n, int_data ;
12    char char_data ;
13    bool success ;
```

- The data type of the stack data is specified between < and >.
- Line 9 generates a stack class from the class template to process up to ten integer data.
- Line 10 generates a stack class from the class template to process up to five character data items.
- Technically, a class generated from a class template is known as a template class.

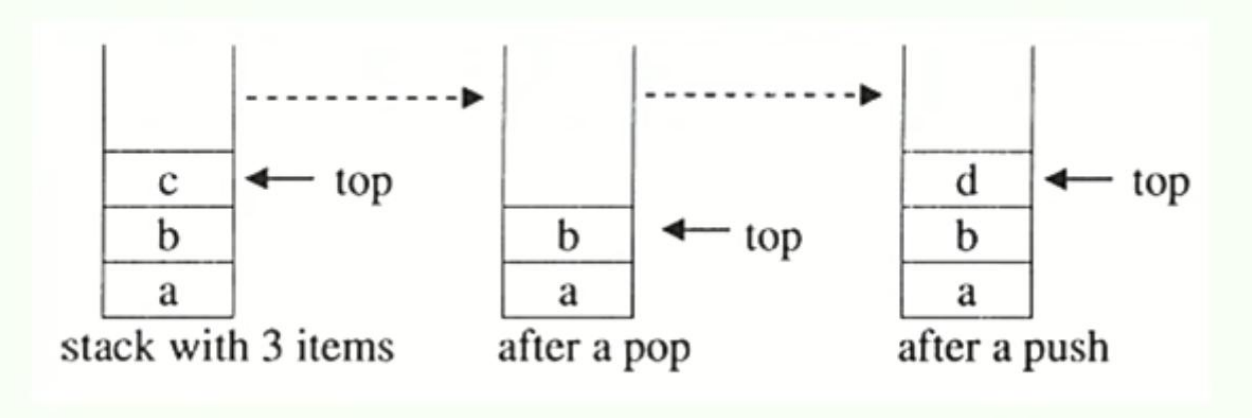
```
15   // Push some values onto c_stack.
16   c_stack.push( 'a' ) ;
17   c_stack.push( 'b' ) ;
18   c_stack.push( 'c' ) ;
19
20   // Use a for loop to fill i_stack.
21   n = i_stack.stack_size() ;
22   for( i = 0 ; i < n ; i++ )
23     i_stack.push( i ) ;
24
25   // Display the values on c_stack.
26   cout << "character stack data:" << endl ;
27   n = c_stack.number_stacked() ;
28   for ( i = 0 ; i < n ; i++ )
29   {
30     success = c_stack.pop( char_data ) ;
31     if ( success )
32       cout << char_data << ' ' ;
33   }
```

```
34   cout << endl ;
35
36   // Display the values on i_stack.
37   cout << "integer stack data:" << endl ;
38   n = i_stack.number_stacked() ;
39   for ( i = 0 ; i < n ; i++ )
40   {
41     success = i_stack.pop ( int_data ) ;
42     if ( success )
43       cout << int_data << ' ' ;
44   }
45   cout << endl ;
46 }
```

- Lines 16 to 18 and lines 22 to 23 push some sample data items onto each of the stack objects c_stack and i_stack.
- Lines 28 to 33 and lines 39 to 44 pop and display data from each of the stacks.

11.3 Template

- Two principal operations for a stack data structure.
  - push: adds an item to the top of the stack.
  - pop: retrieves the most recently pushed item from the stack.



a stack of plates, whereby plates are only added and removed from the top of the stack

- A simple way to implement a stack is to use an array to hold the items and a variable to hold the index value of the 'top' of the array.

11.3 Template

- To create templates, first write a non-template data type specific version of the function or class.
- When the data type specific version is working satisfactorily replace the specific data types with template parameters

11.3 Template

Class Templates Example

```
#include <iostream>
using namespace std;

template <class T>
class Number {
  private:
   // Variable of type T
   T num;

  public:
   // constructor
   Number(T n) : num(n) {}

   T getNum() {
      return num;
   }
};
```

```
int main() {

    // create object with int type
    Number<int> numberInt(7);

    // create object with double type
    Number<double> numberDouble(7.7);

    cout << "int Number = " << numberInt.getNum() << endl;
    cout << "double Number = " << numberDouble.getNum() << endl;

    return 0;
}
```

11.4 Virtual functions

- C++ uses **virtual functions** to implement dynamic binding.
- Virtual functions allow polymorphism to work in all situations.
- The keyword **virtual** is optional in the derived classes, although it is probably a good idea to include it for the sake of clarity.

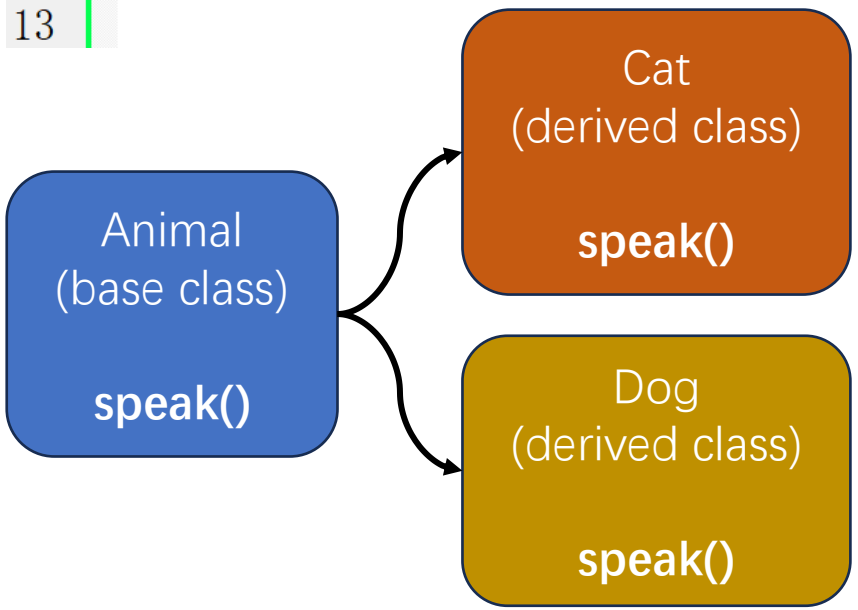
11.4 Virtual functions

```cpp
#include <iostream>
#include <string>

using namespace std;

class Animal
{
public:
    Animal() {};

    string speak() { return "???"; }
};
```




```cpp
class Cat: public Animal
{
public:
    Cat() {};

    string speak() { return "Meow"; }
};

class Dog: public Animal
{
public:
    Dog() {};

    string speak() { return "Woof"; }
};

void Says(Animal& animal)
{
    cout << animal.speak() << '\n';
}
```

11.4 Virtual functions

```
30  void Says(Animal& animal)
31  {
32      cout << animal.speak() << '\n';
33  }
34
35  int main()
36  {
37      Animal myAnimal;
38      Cat myCat;
39      Dog myDog;
40
41      cout << "My animal (base class) says ";
42      Says(myAnimal);
43
44      cout << "My cat says ";
45      Says(myCat);
46
47      cout << "My dog says ";
48      Says(myDog);
49  }
50
```

Code execution result:

```
My animal (base class) says ???
My cat says ???
My dog says ???
```

11.4 Virtual functions

```
6  class Animal
7  {
8  public:
9      Animal(){};
10
11     virtual string speak() { return "???"; }
12 };
13
14 class Cat: public Animal
15 {
16 public:
17     Cat(){};
18
19     string speak() { return "Meow"; }
20 };
21
22 class Dog: public Animal
23 {
24 public:
25     Dog(){};
26
27     string speak() { return "Woof"; }
28 };
29
30 void Says(Animal& animal)
31 {
32     cout << animal.speak() << '\n';
33 }
34
```

```
35 int main()
36 {
37     Animal myAnimal;
38     Cat myCat;
39     Dog myDog;
40
41     cout << "My animal (base class) says ";
42     Says(myAnimal);
43
44     cout << "My cat says ";
45     Says(myCat);
46
47     cout << "My dog says ";
48     Says(myDog);
49 }
50
```

Code execution result:

```
My animal (base class) says ???
My cat says Meow
My dog says Woof
```

11.4 Virtual functions

- **When to use virtual functions**
  - There are memory and execution time overheads associated with virtual functions, so be judicious in the choice of whether a base class function is virtual or not.
  - In general, declare a base class member function as **virtual** if it **may be overridden in a derived class.**
- **Overriding and overloading**
  - **Overloading** is a compiler technique that distinguishes between **functions with the same name** but with different parameter lists. Function overloading is covered in lecture 8.
  - **Overriding** occurs in inheritance when **a member function of a derived class has the same name and the same parameters as a member function of its base class.**

11.5 Abstract base classes

- A base class that contains a pure virtual function is called an **abstract base class**.
  - The base class member function has no code and is assigned to 0

```
virtual void display_details() = 0 ;
```

  - A class member function defined in this way is called a ***pure virtual function***.
  - A pure virtual function is required by the compiler but doesn't actually do anything.
  - A pure virtual function is overridden in all the derived classes and hence there is no need to implement it.

11.5 Abstract base classes

- **Abstract base classes** are not used to create objects, but exist **solely as a base for deriving other classes**.

- It is **not** possible to define an object of an **abstract base class**, i.e. objects of an abstract base class **cannot be instantiated**.

```
class Animal    //Abstract base class
{
 public :

 virtual void speak() = 0; //Pure virtual Function
};
```

```
Animal myAnimal; // error : variable of an abstract class
```

- Although a class hierarchy does not have to contain an abstract base class, typically one is defined during the first stage of developing a class hierarchy.